

June 29, 2026

# Stork Oracle

## Smart Contract Security Assessment

Placeholder text for the main body of the report.

## Contents

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>About Zellic</b>                                           | <b>4</b>  |
| <hr/>                                                         |           |
| <b>1. Overview</b>                                            | <b>5</b>  |
| 1.1. Executive Summary                                        | 5         |
| 1.2. Goals of the Assessment                                  | 5         |
| 1.3. Non-goals and Limitations                                | 5         |
| 1.4. Results                                                  | 5         |
| <hr/>                                                         |           |
| <b>2. Introduction</b>                                        | <b>7</b>  |
| 2.1. About Stork Oracle                                       | 7         |
| 2.2. Methodology                                              | 7         |
| 2.3. Scope                                                    | 9         |
| 2.4. Project Overview                                         | 9         |
| 2.5. Project Timeline                                         | 10        |
| <hr/>                                                         |           |
| <b>3. Detailed Findings</b>                                   | <b>11</b> |
| 3.1. Reinitialization can create multiple shared Stork states | 11        |
| 3.2. Metadata hash fields are not length validated            | 13        |
| 3.3. Stale single updates consume fees                        | 16        |
| 3.4. Admin capability cannot be rotated                       | 19        |
| 3.5. i128 sign comment is inverted                            | 22        |
| 3.6. Stale-only update batches do not revert                  | 24        |
| 3.7. Insufficient test coverage                               | 27        |

---

|           |                               |           |
|-----------|-------------------------------|-----------|
| <b>4.</b> | <b>Discussion</b>             | <b>30</b> |
| 4.1.      | Stork Sui address can be zero | 30        |

---

|           |                                            |           |
|-----------|--------------------------------------------|-----------|
| <b>5.</b> | <b>Threat Model</b>                        | <b>32</b> |
| 5.1.      | Component: stork                           | 32        |
| 5.2.      | Component: state                           | 34        |
| 5.3.      | Component: verify                          | 37        |
| 5.4.      | Component: Update inputs and encoded types | 39        |

---

|           |                           |           |
|-----------|---------------------------|-----------|
| <b>6.</b> | <b>Assessment Results</b> | <b>43</b> |
| 6.1.      | Disclaimer                | 43        |

## About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website [zellic.io](https://zellic.io) and follow [@zellic\\_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at [hello@zellic.io](mailto:hello@zellic.io).



## 1. Overview

### 1.1. Executive Summary

Zellic conducted a security assessment for Stork Network from June 26th to June 29th, 2026. During this engagement, Zellic reviewed Stork Oracle's code for security vulnerabilities, design issues, and general weaknesses in security posture.

---

### 1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Are there any permissioning oversights that would allow unauthorized parties to perform privileged actions, such as modifying oracle configuration or withdrawing update fees?
  - Could EVM signature validation be bypassed, allowing an attacker to submit forged price updates?
  - Could negative, large, or unexpected numeric values be mishandled when encoding, verifying, or storing the temporal numeric price feeds?
  - Is the contract upgrade and migration path compatible across versions, and are permissions around upgradability correctly enforced?
- 

### 1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

---

### 1.4. Results

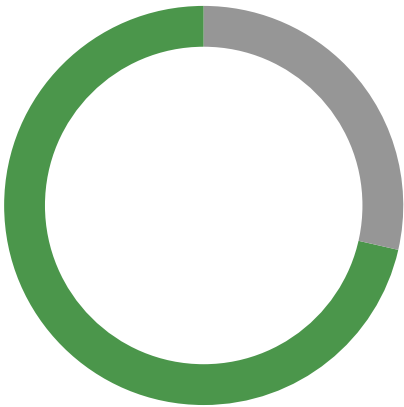
During our assessment on the scoped Stork Oracle modules, we discovered seven findings. No critical issues were found. Five findings were of low impact and the remaining findings were informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Stork

Network in the Discussion section ([4](#), [7](#)).

Breakdown of Finding Impacts

| Impact Level             | Count |
|--------------------------|-------|
| <div>Critical</div>      | 0     |
| <div>High</div>          | 0     |
| <div>Medium</div>        | 0     |
| <div>Low</div>           | 5     |
| <div>Informational</div> | 2     |



## 2. Introduction

### 2.1. About Stork Oracle

Stork Network contributed the following description of Stork Oracle:

Stork is an oracle project that primarily serves perpetual futures decentralized exchanges. Stork collects signed prices from a variety of publishers and aggregates the data to a single signed canonical price on a regular cadence.

These prices are made available off-chain via an auth-gated WebSocket server. Clients can consume these prices off-chain, and then opt into posting data on-chain via the Stork contract for a variety of chains. On-chain applications can then use these verified prices as needed for their application-specific needs like trade executions, liquidations, etc.

---

### 2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

**Basic coding mistakes.** Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the modules.

**Business logic errors.** Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

**Integration risks.** Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

**Code maturity.** We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped modules itself. These observations — found in the Discussion ([4.7](#)) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.



## 2.3. Scope

The engagement involved a review of the following targets:

### Stork Oracle Modules

|            |                                                                                                               |
|------------|---------------------------------------------------------------------------------------------------------------|
| Type       | Move                                                                                                          |
| Platform   | Sui                                                                                                           |
| Target     | stork-external                                                                                                |
| Repository | <a href="https://github.com/Stork-Oracle/stork-external">https://github.com/Stork-Oracle/stork-external</a> ↗ |
| Version    | c487e9bdcd2a0fe9eb5a57c8e61a54ac4a3795de                                                                      |
| Programs   | chains/sui/contracts/sources/*.move                                                                           |

## 2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of two person-days. The assessment was conducted by two consultants over the course of two calendar days.

## Contact Information

---

The following project manager was associated with the engagement:

**Pedro Moura**  
✈ Engagement Manager  
[pedro@zellic.io](mailto:pedro@zellic.io) ↗

---

The following consultants were engaged to conduct the assessment:

**Nipun Gupta**  
✈ Engineer  
[nipun@zellic.io](mailto:nipun@zellic.io) ↗

**Jisub Kim**  
✈ Engineer  
[jisub@zellic.io](mailto:jisub@zellic.io) ↗

---

## 2.5. Project Timeline

The key dates of the engagement are detailed below.

**June 26, 2026** Start of primary review period

---

**June 29, 2026** End of primary review period

### 3. Detailed Findings

#### 3.1. Reinitialization can create multiple shared Stork states

|                   |                   |                 |     |
|-------------------|-------------------|-----------------|-----|
| <b>Target</b>     | stork             |                 |     |
| <b>Category</b>   | Integration Risks | <b>Severity</b> | Low |
| <b>Likelihood</b> | Low               | <b>Impact</b>   | Low |

#### Description

The Sui initializer can be called more than once by any holder of the AdminCap. Each call creates and shares a new StorkState object with its own feed registry and treasury and then emits a new initialization event. There is no package-level guard that records initialization or prevents a second canonical state from being created.

The `init_stork` function only requires an AdminCap, and every successful call constructs a fresh shared state:

```
entry fun init_stork(
    _: &AdminCap,
    stork_sui_address: address,
    stork_evm_public_key: vector<u8>,
    single_update_fee: u64,
    stork_state_version: u64,
    ctx: &mut TxContext,
) {
    let state = state::new(stork_sui_address, stork_evm_public_key,
        single_update_fee, stork_state_version, ctx);
    let state_id = object::id(&state);
    state::share(state);
    event::emit_stork_initialization_event(
        stork_sui_address,
        stork_evm_public_key,
        single_update_fee,
        state_id,
        stork_state_version
    );
}
```

By contrast, the EVM deployment path uses OpenZeppelin's `initializer` modifier, so the proxy initializer can only be executed once:

```
function initialize(
```

```
address initialOwner,  
address storkPublicKey,  
uint validTimePeriodSeconds,  
uint singleUpdateFeeInWei  
) initializer public {  
    __Ownable_init(initialOwner);  
    __UUPSUpgradeable_init();  
  
    _initialize(storkPublicKey, validTimePeriodSeconds, singleUpdateFeeInWei);  
}
```

## Impact

This issue is admin-gated, so it does not provide a permissionless attack path. It can still cause deployment and integration failures if `init_stork` is accidentally called more than once, because clients may read from or push updates to different state objects. Fees and feed values would then be split across independent registries.

## Recommendations

Store a singleton initialization marker, or otherwise make `init_stork` consume a one-time capability so only one shared `StorkState` can be created for the package.

## Remediation

Stork Network has acknowledged this issue and provided the following statement:

The `init` function is admin-gated, and canonical state is already established. Any second state is inert/an orphan. This issue does not directly correlate to our EVM contract as a re-initialization would not allow for a replacement of the existing stork state.

### 3.2. Metadata hash fields are not length validated

|                   |                 |                 |     |
|-------------------|-----------------|-----------------|-----|
| <b>Target</b>     | verify          |                 |     |
| <b>Category</b>   | Coding Mistakes | <b>Severity</b> | Low |
| <b>Likelihood</b> | Medium          | <b>Impact</b>   | Low |

#### Description

The verifier accepts `publisher_merkle_root` and `value_compute_alg_hash` as arbitrary `vector<u8>` values. These fields are intended to mirror the EVM contract's `bytes32 publisherMerkleRoot` and `bytes32 valueComputeAlgHash` fields, but the Sui code does not enforce a 32-byte length for either value.

The update payload constructor stores both metadata fields without validating their sizes:

```
public fun new(
    id: vector<u8>,
    temporal_numeric_value_timestamp_ns: u64,
    temporal_numeric_value_magnitude: u128,
    temporal_numeric_value_negative: bool,
    publisher_merkle_root: vector<u8>,
    value_compute_alg_hash: vector<u8>,
    r: vector<u8>,
    s: vector<u8>,
    v: u8,
): UpdateTemporalNumericValueEvmInput {
    UpdateTemporalNumericValueEvmInput {
        id: encoded_asset_id::from_bytes(id),
        temporal_numeric_value:
            temporal_numeric_value::new(temporal_numeric_value_timestamp_ns,
            i128::new(temporal_numeric_value_magnitude,
            temporal_numeric_value_negative)),
        publisher_merkle_root,
        value_compute_alg_hash,
        r,
        s,
        v,
    }
}
```

The signature message then appends the two vectors directly, without a delimiter or length prefix:

```
fun get_stork_message(
    stork_evm_public_key: &EvmPubkey,
    id: vector<u8>,
    recv_time: u64,
    quantized_value: I128,
    publisher_merkle_root: vector<u8>,
    value_compute_alg_hash: vector<u8>,
): vector<u8> {
    let mut data = vector::empty<u8>();
    vector::append(&mut data, evm_pubkey::get_bytes(stork_evm_public_key));
    vector::append(&mut data, id);
    // [...]
    vector::append(&mut data, publisher_merkle_root);
    vector::append(&mut data, value_compute_alg_hash);
    data
}
```

This means the signature binds to the concatenated data, not to the boundary between `publisher_merkle_root` and `value_compute_alg_hash`. Therefore, a signature over `root || alg_hash` can also validate for a payload that presents `root || alg_hash[0]` as `publisher_merkle_root` and `alg_hash[1..]` as `value_compute_alg_hash`, because the hashed bytes are unchanged.

The current feed storage only persists the asset ID and numeric value, so malformed metadata splits do not directly corrupt stored feed data. However, the verifier still accepts payloads whose metadata fields do not match the fixed-width EVM format, and future consumers that rely on these fields or related events may receive values that were not individually authenticated.

## Impact

Malformed metadata splits can pass signature verification on Sui while they would not be representable as the corresponding EVM `bytes32` fields. This weakens cross-chain consistency and can create integration risk for consumers that expect the signature to authenticate a distinct 32-byte publisher Merkle root and a distinct 32-byte value-computation algorithm hash.

## Recommendations

Validate that both metadata fields are exactly 32 bytes before constructing or verifying an update payload.

```
const EInvalidMetadataLength: u64 = 2;

public fun new(
    id: vector<u8>,
```

```
temporal_numeric_value_timestamp_ns: u64,  
temporal_numeric_value_magnitude: u128,  
temporal_numeric_value_negative: bool,  
publisher_merkle_root: vector<u8>,  
value_compute_alg_hash: vector<u8>,  
r: vector<u8>,  
s: vector<u8>,  
v: u8,  
) : UpdateTemporalNumericValueEvmInput {  
    assert!(publisher_merkle_root.length() == 32, EInvalidMetadataLength);  
    assert!(value_compute_alg_hash.length() == 32, EInvalidMetadataLength);  
    UpdateTemporalNumericValueEvmInput {  
        id: encoded_asset_id::from_bytes(id),  
        temporal_numeric_value:  
temporal_numeric_value::new(temporal_numeric_value_timestamp_ns,  
i128::new(temporal_numeric_value_magnitude,  
temporal_numeric_value_negative)),
```

## Remediation

This issue has been acknowledged by Stork Network, and a fix was implemented in commit [ae4d5621](#).

### 3.3. Stale single updates consume fees

|                   |                |                 |     |
|-------------------|----------------|-----------------|-----|
| <b>Target</b>     | stork          |                 |     |
| <b>Category</b>   | Business Logic | <b>Severity</b> | Low |
| <b>Likelihood</b> | High           | <b>Impact</b>   | Low |

#### Description

The Sui single-update path deposits the submitted fee before checking whether the submitted timestamp is fresher than the existing feed value. If the feed already contains a value with an equal or greater timestamp, the function returns successfully without updating the feed, but the caller's fee has already been joined into the protocol treasury.

The fee is accepted before the stale-value check:

```
public fun update_single_temporal_numeric_value_evm(
    stork_state: &mut StorkState,
    update_data: UpdateTemporalNumericValueEvmInput,
    fee: Coin<SUI>,
    ctx: &mut TxContext,
) {
    let feed_id = update_data.get_id();
    let evm_pubkey = stork_state.get_stork_evm_public_key();
    let fee_in_mist = stork_state.get_single_update_fee_in_mist();

    assert!(fee.value() >= fee_in_mist, EInsufficientFee);
    stork_state.deposit_fee(fee);

    let feed_registry = stork_state.borrow_tnv_feeds_registry_mut();

    if (feed_registry.contains(feed_id)) {
        let feed = feed_registry.borrow_mut(feed_id);
        if (feed.get_latest_canonical_temporal_numeric_value_unchecked().
            get_timestamp_ns() >=
            update_data.get_temporal_numeric_value().get_timestamp_ns()) {
            return
        }
    }
};

// [...]
create_or_update_temporal_numeric_value_feed(feed_registry, feed_id,
update_data, ctx);
```



```
}
```

The deposit operation joins the full provided coin into the treasury:

```
public fun deposit_fee(stork_state: &mut StorkState, fee: Coin<SUI>) {  
    assert!(stork_state.version == VERSION, EIncorrectVersion);  
    let treasury = dynamic_object_field::borrow_mut(  
        &mut stork_state.id,  
        TREASURY_NAME,  
    );  
    coin::join(treasury, fee)  
}
```

The EVM update path reverts when no submitted value is fresh, which refunds `msg.value` as part of the reverted transaction:

```
function _validateUpdatesAndFee(uint16 numUpdates) private {  
    if (numUpdates == 0) revert StorkErrors.NoFreshUpdate();  
    uint requiredFee = getTotalFee(numUpdates);  
    if (msg.value < requiredFee) revert StorkErrors.InsufficientFee();  
}
```

As a result, the Sui path can charge callers for a no-op stale update, while the EVM path rejects the same condition.

## Impact

Callers and off-chain pushers can lose the submitted update fee when a single update races with a fresher value or is retried after the feed has already advanced. The affected amount is limited to the fee attached to the stale update, but the transaction succeeds without producing the expected feed update.

## Recommendations

Move the fee deposit after the freshness check, and abort when the submitted update is not fresher than the stored value.

```
const EInvalidSignature: u64 = 0;  
const EInsufficientFee: u64 = 1;  
const EFeedNotFound: u64 = 2;  
const ENoFreshUpdate: u64 = 3;  
  
// [...]
```

```

    assert!(fee.value() >= fee_in_mist, EInsufficientFee);
    stork_state.deposit_fee(fee);

    let feed_registry = stork_state.borrow_tnv_feeds_registry_mut();

    if (feed_registry.contains(feed_id)) {
        let feed = feed_registry.borrow_mut(feed_id);
        if (feed.get_latest_canonical_temporal_numeric_value_unchecked().
            get_timestamp_ns() >=
update_data.get_temporal_numeric_value().get_timestamp_ns()) {
            return
            abort ENoFreshUpdate
        }
    };

    // [...]
    create_or_update_temporal_numeric_value_feed(feed_registry, feed_id,
update_data, ctx);
    stork_state.deposit_fee(fee);

```

## Remediation

This issue has been acknowledged by Stork Network, and a fix was implemented in commit [ea3a04a7](#).

### 3.4. Admin capability cannot be rotated

|                   |                   |                 |     |
|-------------------|-------------------|-----------------|-----|
| <b>Target</b>     | admin             |                 |     |
| <b>Category</b>   | Integration Risks | <b>Severity</b> | Low |
| <b>Likelihood</b> | Low               | <b>Impact</b>   | Low |

#### Description

The Sui package creates a single AdminCap during module initialization, but it does not expose any function to transfer or rotate that capability. The AdminCap type has key but not store, which means generic public transfer flows are not available outside the defining module. Since the admin module only transfers the capability to the package publisher during initialization, the package has no in-protocol equivalent of the EVM contract's two-step ownership transfer.

The admin module defines AdminCap without store and transfers it only during initialization:

```
public struct AdminCap has key{
    id: UID,
}

fun init(ctx: &mut TxContext) {
    transfer::transfer(
        new(ctx),
        ctx.sender()
    );
}

fun new(ctx: &mut TxContext): AdminCap {
    AdminCap { id: object::new(ctx) }
}
```

All privileged Sui functions require a reference to this capability:

```
public fun update_single_update_fee_in_mist(
    _: &AdminCap,
    state: &mut StorkState,
    new_single_update_fee_in_mist: u64,
) {
    assert!(state.version == VERSION, EIncorrectVersion);
    state.single_update_fee_in_mist = new_single_update_fee_in_mist;
}
```

The EVM contract uses Ownable2StepUpgradeable, initializes an owner, and therefore inherits an ownership rotation flow through transferOwnership and acceptOwnership:

```
contract UpgradeableStork is Initializable, UUPSUpgradeable,
    Ownable2StepUpgradeable, Stork {
    function initialize(
        address initialOwner,
        address storkPublicKey,
        uint validTimePeriodSeconds,
        uint singleUpdateFeeInWei
    ) initializer public {
        __Ownable_init(initialOwner);
        __UUPSUpgradeable_init();

        _initialize(storkPublicKey, validTimePeriodSeconds,
            singleUpdateFeeInWei);
    }

    function updateValidTimePeriodSeconds(uint validTimePeriodSeconds)
    public onlyOwner override {
        setValidTimePeriodSeconds(validTimePeriodSeconds);
    }
}
```

This creates an operational mismatch between the EVM and Sui deployments. On Sui, the initial AdminCap holder remains the only administrator unless a package upgrade introduces a rotation path.

## Impact

If the initial admin account must be replaced, migrated to a new custody setup, or moved to a multi-sig after deployment, there is no package-level function to do so. Loss of access to the original AdminCap would permanently remove the ability to update fees, update signing metadata, migrate state, or withdraw collected fees.

## Recommendations

Add an admin-only function that transfers AdminCap to a new owner, or define the capability with the abilities needed for the intended custody model. If the capability should remain nontransferable outside the module, document that explicitly and provide an upgrade or recovery procedure for admin custody changes.

## Remediation

This issue has been acknowledged by Stork Network, and a fix was implemented in commit [c34f2fd3](#).

### 3.5. i128 sign comment is inverted

|                   |               |                 |               |
|-------------------|---------------|-----------------|---------------|
| <b>Target</b>     | i128          |                 |               |
| <b>Category</b>   | Code Maturity | <b>Severity</b> | Informational |
| <b>Likelihood</b> | N/A           | <b>Impact</b>   | Informational |

#### Description

The comment describing the negative field of i128 states the opposite of the implementation. The surrounding examples and all accessors treat `negative == true` as a negative value and `negative == false` as a nonnegative value, but the field comment says true means positive.

The struct documentation is internally inconsistent:

```
// the magnitude is the absolute value of the number
// positive 1 is represented as (1, false)
// negative 1 is represented as (1, true)
public struct I128 has copy, drop, store {
    // sign of the i128, True if positive, false if negative
    negative: bool,
    // magnitude of the i128
    magnitude: u128,
}
```

The implementation confirms that the field is a negative flag:

```
public fun is_negative(i128: &I128): bool {
    i128.negative
}
```

#### Impact

There is no direct security impact. The inverted comment can mislead maintainers and integrators when constructing signed values or reviewing signature-encoding logic.

#### Recommendations

Update the field comment so it matches the implementation.

```
public struct I128 has copy, drop, store {  
    // sign of the i128, True if positive, false if negative  
    // sign of the i128, true if negative, false if positive  
    negative: bool,  
    // magnitude of the i128  
    magnitude: u128,  
}
```

## Remediation

This issue has been acknowledged by Stork Network, and a fix was implemented in commit [5a8f1882](#).

### 3.6. Stale-only update batches do not revert

|                   |                |                 |               |
|-------------------|----------------|-----------------|---------------|
| <b>Target</b>     | stork          |                 |               |
| <b>Category</b>   | Business Logic | <b>Severity</b> | Informational |
| <b>Likelihood</b> | N/A            | <b>Impact</b>   | Informational |

#### Description

The Sui batch-update path accepts a batch whose entries are all stale relative to the stored feed values. The function skips stale entries by continuing the loop, leaves num\_updates at zero, computes a zero required fee, and then deposits the caller-provided fee without signaling that no feed was updated.

The Sui implementation only checks that the paid fee covers num\_updates, even when num\_updates is zero:

```
public fun update_multiple_temporal_numeric_values_evm(
    stork_state: &mut StorkState,
    update_data: UpdateTemporalNumericValueEvmInputVec,
    fee: Coin<SUI>,
    ctx: &mut TxContext,
) {
    let evm_pubkey = stork_state.get_stork_evm_public_key();
    let feed_registry = stork_state.borrow_tnv_feeds_registry_mut();
    let mut num_updates = 0;

    let mut i = 0;
    while (i < update_data.length()) {
        let update = update_data.get_data()[i];
        let feed_id = update.get_id();
        if (feed_registry.contains(feed_id)) {
            let feed = feed_registry.borrow_mut(feed_id);
            if (feed.get_latest_canonical_temporal_numeric_value_unchecked().
                get_timestamp_ns() >=
            update.get_temporal_numeric_value().get_timestamp_ns()) {
                i = i + 1;
                continue
            };
        };

        // [...]
        create_or_update_temporal_numeric_value_feed(feed_registry, feed_id,
```



```
update, ctx);
    num_updates = num_updates + 1;
};
let required_fee = stork_state.get_total_fees_in_mist(num_updates);
assert!(fee.value() >= required_fee, EInsufficientFee);
stork_state.deposit_fee(fee);
}
```

The analogous EVM implementation rejects this case after processing the batch:

```
function _validateUpdatesAndFee(uint16 numUpdates) private {
    if (numUpdates == 0) revert StorkErrors.NoFreshUpdate();
    uint requiredFee = getTotalFee(numUpdates);
    if (msg.value < requiredFee) revert StorkErrors.InsufficientFee();
}
```

As a result, the Sui function can return successfully even though it performed no oracle update. This diverges from the EVM contract behavior and makes success ambiguous for callers and off-chain pushers.

## Impact

Callers can observe a successful transaction for a batch that did not update any feed. If the caller attaches a nonzero fee, that fee is deposited even though all updates were stale.

## Recommendations

Track whether at least one feed was updated, and abort when `num_updates` is zero before accepting the payment.

```
const EInvalidSignature: u64 = 0;
const EInsufficientFee: u64 = 1;
const EFeedNotFound: u64 = 2;
const ENoFreshUpdate: u64 = 3;

// [...]

};

assert!(num_updates > 0, ENoFreshUpdate);
let required_fee = stork_state.get_total_fees_in_mist(num_updates);
assert!(fee.value() >= required_fee, EInsufficientFee);
stork_state.deposit_fee(fee);
```

## Remediation

This issue has been acknowledged by Stork Network, and a fix was implemented in commit [ea3a04a7](#).

### 3.7. Insufficient test coverage

|                   |               |                 |     |
|-------------------|---------------|-----------------|-----|
| <b>Target</b>     | Project-wide  |                 |     |
| <b>Category</b>   | Code Maturity | <b>Severity</b> | Low |
| <b>Likelihood</b> | N/A           | <b>Impact</b>   | Low |

#### Description

The package has a working test suite. It covers the main success paths and the core building blocks. The signature verifier, the `i128` math, and the encoded ID and public key types each have unit tests. The end-to-end success paths are tested too: a single update, a negative value update, a batch with one entry, setup, and fee withdrawal.

Several branches and failure cases, however, are not tested. A few of these untested paths coincide with issues identified elsewhere in this report, which suggests that closing the gaps would add meaningful assurance.

The missing tests fall into a few groups.

#### Fee and freshness logic in stork

- The single update case is not tested; `update_single_temporal_numeric_value_evm` takes the fee before it checks freshness then returns early if the new value is not newer, so the caller pays but nothing is written. Please see Finding [3.3](#), ↗ for more detail.
- The case where every batch entry is stale is not tested. When every entry in `update_multiple_temporal_numeric_values_evm` is stale, the fee is computed as zero but the caller's coin is still kept. Please see Finding [3.6](#), ↗ for more detail.
- Batches with more than one entry, and batches that mix fresh and stale entries, are not tested. Only a batch with one entry is tested, so the fee math per entry and the skip loop are unchecked.
- The `EInsufficientFee` case, where the paid coin is too small, is not tested on either path.

#### Setup, admin, and upgrade paths in stork and state

- Calling `init_stork` twice is not tested. A second call creates another shared `StorkState`. Please see Finding [3.1](#), ↗ for more detail.
- The `migrate` function is not tested at all, neither the success case nor the abort on a wrong version.
- The `withdraw_fees` function on an empty treasury, which should abort with `ENoFeesToWithdraw`, is not tested.
- The version check, which aborts with `EIncorrectVersion`, is not tested on any getter, setter, or update.

### Negative signature cases in verify

- A signature of the wrong length, a v other than 27 or 28, a reshaped signature with a high s value, and a signature checked against the wrong key are not tested. These cases exercise the signature verification boundary most directly.

### Input checks and limits

- A publisher\_merkle\_root or value\_compute\_alg\_hash that is not 32 bytes and an EVM key of the wrong length passed to update\_stork\_evm\_public\_key are not tested. Please see Finding [3.2](#) ↗ for more detail.
- Setting stork\_sui\_address to the zero address is not tested.

## Impact

Without these tests, the following may occur:

- Issues may reach production undetected. During this review, the untested fee and freshness branches and the missing reinitialization test coincided with issues identified through manual review.
- Small changes or refactors can quietly break existing behavior, especially around fees and the staleness check.
- Core assumptions, such as the version check, the rule that only one state exists, and the signature gate, can break without any test failing.

Good tests make the code easier to trust and to audit, and they lower the cost of fixing problems after launch.

## Recommendations

Add tests for the gaps above, in this order.

1. **Fees and freshness first.** Test a stale single update, and check that the fee was taken with no write. Test a batch where every entry is stale, and check the fee and the state. Test batches with multiple entries and batches that mix fresh and stale entries, and check the fee math per entry. Test that a coin that is too small aborts with EInsufficientFee.
2. **Admin and upgrade paths next.** Test a second init\_stork call, migrate in both the success case and the wrong version case, withdraw\_fees on an empty treasury, and the EIncorrectVersion abort on a stale StorkState.
3. **Negative signature cases.** Test a signature of the wrong length, a bad v, a signature with a high s value, and a wrong key.
4. **Input checks and invariants.** Test metadata that is not 32 bytes, an EVM key of the wrong length, and a zero stork\_sui\_address. Also add checks that only one state

exists, that feed timestamps only move forward, and that functions guarded by the version check abort on the wrong version.

Specific missing test cases include but are not limited to those documented in section [5](#). ↗.

## Remediation

This issue has been acknowledged by Stork Network, and fixes were implemented in the following commits:

- [02969d63](#) ↗
- [7dc323fe](#) ↗

## 4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

### 4.1. Stork Sui address can be zero

The Sui initialization path accepts `stork_sui_address` without validating that it is nonzero. The value is stored in `StorkState` and emitted in the initialization event, so an accidental zero address would become part of the published state metadata.

The `init_stork` function forwards `stork_sui_address` directly into `state::new` and the initialization event:

```
entry fun init_stork(
  _: &AdminCap,
  stork_sui_address: address,
  stork_evm_public_key: vector<u8>,
  single_update_fee: u64,
  stork_state_version: u64,
  ctx: &mut TxContext,
) {
  let state = state::new(stork_sui_address, stork_evm_public_key,
    single_update_fee, stork_state_version, ctx);
  let state_id = object::id(&state);
  state::share(state);
  event::emit_stork_initialization_event(
    stork_sui_address,
    stork_evm_public_key,
    single_update_fee,
    state_id,
    stork_state_version
  );
}
```

The state constructor only checks the version then stores the provided address:

```
public(package) fun new(
  stork_sui_address: address,
  stork_evm_public_key: vector<u8>,
  single_update_fee_in_mist: u64,
  version: u64,
  ctx: &mut TxContext,
): StorkState {
```

```
assert!(version == VERSION, EIncorrectVersion);

let evm_pubkey = evm_pubkey::from_bytes(stork_evm_public_key);

// [...]

StorkState {
    id: uid,
    stork_sui_address,
    stork_evm_public_key: evm_pubkey,
    single_update_fee_in_mist,
    version,
}
}
```

The same pattern exists in `update_stork_sui_address`, which lets the admin replace the stored Sui address with any address value:

```
public fun update_stork_sui_address(
    _: &AdminCap,
    state: &mut StorkState,
    new_stork_sui_address: address,
) {
    assert!(state.version == VERSION, EIncorrectVersion);
    state.stork_sui_address = new_stork_sui_address;
}
```

The address is configuration metadata, and the relevant functions are admin-gated. However, accepting the zero address can make deployment mistakes harder to detect and can cause off-chain consumers that rely on `stork_sui_address` or `StorkInitializationEvent` to ingest invalid metadata. If this address is expected to identify a real Stork-owned Sui account, validating that it is nonzero during initialization and admin updates would make the deployment invariant explicit.

This issue has been acknowledged by Stork Network.

## 5. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the modules and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

### 5.1. Component: stork

#### Description

The stork module is the main entry point of the protocol. It has two functions that write prices on chain, one function that reads a price back, and one function that sets the protocol up.

The `update_single_temporal_numeric_value_evm` function updates a single feed. It first reads the update fee and the Stork EVM public key from the shared `StorkState`. It takes the fee, and only then decides whether the update is worth writing:

```
assert!(fee.value() >= fee_in_mist, EInsufficientFee);
stork_state.deposit_fee(fee);

let feed_registry = stork_state.borrow_tnv_feeds_registry_mut();

if (feed_registry.contains(feed_id)) {
    let feed = feed_registry.borrow_mut(feed_id);
    if (feed.get_latest_canonical_temporal_numeric_value_unchecked().
        get_timestamp_ns() >=
        update_data.get_temporal_numeric_value().get_timestamp_ns()) {
        return
    }
};
```

Note the order: the fee is added to the treasury before the freshness check. So if the update turns out to be stale, the caller still pays. The signature is checked only after this point, and a valid value is written through the private `create_or_update_temporal_numeric_value_feed` helper.

The `update_multiple_temporal_numeric_values_evm` function does the same thing for a batch of updates. It loops over the batch, skips any entry that is stale, and writes the rest, counting the writes in `num_updates`. The fee is charged once at the end, based on that count:

```
let required_fee = stork_state.get_total_fees_in_mist(num_updates);
assert!(fee.value() >= required_fee, EInsufficientFee);
stork_state.deposit_fee(fee);
```



If every entry in the batch is stale, `num_updates` stays at zero, the required fee is zero, and the caller's coin is still deposited.

The `get_temporal_numeric_value_unchecked` function reads the latest value for a feed. It aborts with `EFeedNotFound` if the feed does not exist. As the name suggests, it does not check whether the value is stale, so callers have to do that themselves.

The `init_stork` function sets the protocol up. It is an entry function that needs the `AdminCap`. It creates a new `StorkState`, shares it, and emits an event. It does not remember that it already ran, so it can be called again.

The `create_or_update_temporal_numeric_value_feed` function is a private helper. It creates a feed the first time it sees an ID — otherwise it overwrites the stored value and then emits an event. It does no checking of its own and trusts the caller to have already verified the signature and the freshness.

## Invariants

- A feed is only written after a signature that matches the Stork EVM key. This holds. Both update paths check `verify_stork_evm_signature` and abort with `EInvalidSignature` before writing.
- A newer value is never replaced by an older or equally old one. This holds. Both paths compare timestamps with `>=` and skip the write when the stored value is at least as fresh.
- A real update pays at least the configured fee. This holds on both paths. But a stale single update still pays even though nothing is written. See Finding [3.3](#) ↗ for more detail.
- A batch that writes nothing should not quietly take a fee. This does not hold. When every entry is stale, the required fee is zero, but the coin the caller sent is still deposited. See Finding [3.6](#) ↗ for more detail.
- There should only ever be one `StorkState`. This does not hold. The `init_stork` function has no guard, so any `AdminCap` holder can create more. See Finding [3.1](#) ↗ for more detail.

## Test coverage

### Cases covered

- Creating a new feed with a positive value and the negative-value case: `test_feed_operations` and `test_feed_operations_negative_value`.
- An update with a bad signature aborting: `test_feed_operations_invalid_signature`, marked `#[expected_failure]`.
- A batch with one valid entry: `test_multiple_feed_updates`.
- Setup and the resulting state fields: `test_stork_init`.

### Cases not covered

- The stale single-update case, where a second update with an equal or older timestamp returns after the fee was already taken.
- A batch where every entry is stale.
- Batches with more than one entry, and batches that mix fresh and stale entries.
- The `EInsufficientFee` rejection on either path.
- A batch that writes one or more entries and then aborts at the final fee assertion, to confirm every write rolls back.
- The `EFeedNotFound` abort from `get_temporal_numeric_value_unchecked`.
- Calling `init_stork` more than once.
- The `EIncorrectVersion` abort when a stale `StorkState` reaches these functions.

## Attack surface

- The whole update payload comes from the caller: the ID, the timestamp, the value, the Merkle root, the alg hash, and the `r`, `s`, and `v` signature parts. The one thing standing between a caller and a feed write is `verify_stork_evm_signature`, so the integrity of every feed depends on that single check. Without a signature that matches the Stork key, an attacker cannot write anything.
- The only timing trick available to an attacker is replay: resubmitting an old but validly signed update. The `>=` freshness check blocks this, because it refuses anything that is not strictly newer, and it also refuses equal timestamps, so the same value cannot be rewritten.
- The fee coin comes from the caller. No path lets a caller underpay for a real write. But both paths let a caller waste their own money on a no-op: a stale single update and an all-stale batch both keep the coin without writing.
- In the batch path, the signature checks and the writes run before the final fee assertion, but a too-small fee still cannot commit a partial update. A Move transaction is atomic, so the abort at the fee check rolls back every write in the batch. The only thing a caller loses on a failing batch is the gas already spent.
- The `feed_id` passed to `get_temporal_numeric_value_unchecked` is arbitrary, but the function only reads, so the worst case is an abort.
- Only the `AdminCap` holder can call `init_stork`, so the risk here is operational, not an outside attack: a duplicate state or a zero `stork_sui_address`. The zero-address case is discussed in section [4.1.7](#).

## 5.2. Component: state

### Description

The state module defines `StorkState`, the shared object that holds the protocol's settings: the Stork Sui address, the Stork EVM public key, the update fee, and the version. It also owns two attached objects: the feed registry and the SUI treasury.

The `deposit_fee` is public, which means anyone who has a `Coin<SUI>` can call it and add that coin to the treasury:

```
public fun deposit_fee(stork_state: &mut StorkState, fee: Coin<SUI>) {
    assert!(stork_state.version == VERSION, EIncorrectVersion);
    let treasury = dynamic_object_field::borrow_mut(
        &mut stork_state.id,
        TREASURY_NAME,
    );
    coin::join(treasury, fee)
}
```

The `withdraw_fees` function needs the `AdminCap`. It checks that the treasury is not empty, takes the entire balance, and emits an event.

There are three admin-only setters: `update_single_update_fee_in_mist`, `update_stork_sui_address`, and `update_stork_evm_public_key`. The EVM-key setter rechecks the 20-byte length every time, through `evm_pubkey::from_bytes`.

The `migrate` entry function is admin-only and is used during upgrades. It only moves a state forward if it is exactly one version behind and then sets a new address and version and reemits the setup event:

```
entry fun migrate(
    _: &AdminCap,
    state: &mut StorkState,
    version: u64,
    stork_sui_address: address,
) {
    assert!(state.version == VERSION - 1, EIncorrectVersion);
    state.stork_sui_address = stork_sui_address;
    state.version = version;
    ...
}
```

The new version comes straight from the caller with no upper limit, so an admin can set it to anything.

The `new`, `share`, and two `borrow_tnv_feeds_registry` functions are public(package), so only code inside the package can call them. The rest are read-only getters. Every public and package function checks `version == VERSION`, where `VERSION` is 2, before touching the state.

## Invariants

- A state stuck on an old version cannot be read or changed until it is migrated. This

holds. Every public and package function checks `version == VERSION` first.

- Migration only moves a state that is exactly one version behind. This holds. The `migrate` function checks `state.version == VERSION - 1`, so it cannot run again once the state is current.
- The version that `migrate` writes should be a known, sensible value. This does not hold. The `migrate` function writes whatever version the caller passes. A wrong value would lock up the state, because every other function would then fail the `version == VERSION` check.
- Only the admin can withdraw fees, and only when there are some. This holds. The `withdraw_fees` function needs the `AdminCap` and aborts with `ENoFeesToWithdraw` if the treasury is empty.
- It should be possible to hand off or rotate admin control. This does not hold. The capability has key but not store, and there is no function to transfer it. See Finding [3.4](#), ↗ for more detail.
- The Stork Sui address should be a real, nonzero value. This does not hold. It is stored as is, with no zero check. See section [4.1](#), ↗ for more detail.

## Test coverage

### Cases covered

- The three admin setters: `test_all_admin_functions`.
- The `withdraw_fees` function after a fee-paying update, checking the amount taken matches the amount paid: `test_withdraw_fees`.
- The `deposit_fee` function, indirectly, through the update paths.

### Cases not covered

- The `migrate` function, both the success case and the wrong-version abort.
- The `withdraw_fees` function on an empty treasury.
- The `deposit_fee` function called directly by an outside caller.
- The `EIncorrectVersion` abort on any getter or setter.
- An EVM-key update with a wrong-length key.
- Setting the Stork Sui address to zero.
- Extreme fee values set through `update_single_update_fee_in_mist`, such as zero, or a value large enough to overflow the `single_update_fee_in_mist * num_updates` calculation in `get_total_fees_in_mist`.

## Attack surface

- Anyone can call `deposit_fee`. All it does is grow the treasury balance, so the worst case is an unwanted donation. It cannot break the accounting or block withdrawals. It does mean the treasury balance is not purely protocol revenue.
- The admin setters, `withdraw_fees`, and `migrate` all need the `AdminCap`, so an outside

caller cannot reach them. The real risk is that there is only one such capability and it cannot be rotated: if it is lost or stolen, there is no recovery and no two-step handoff.

- The `update_stork_evm_public_key` function is the most sensitive admin action, because it decides whose signatures the update path will accept. A wrong value quietly blocks all future updates; an attacker's value would let the attacker sign any price. The input is length-checked but nothing more.
- The `update_single_update_fee_in_mist` function takes an arbitrary u64. A zero fee makes updates free, and a very large fee can price updates out entirely or make the batch fee calculation `single_update_fee_in_mist * num_updates` in `get_total_fees_in_mist` overflow and abort. Like the other setters, this is an AdminCap risk, not something an outside caller can reach.
- The `migrate` function trusts the version the admin passes, with no limit, so a typo could lock the state until a matching package is deployed. This is an operational risk, not an outside attack.
- The treasury and registry are stored under the fixed names `b"treasury"` and `b"temporal_numeric_value_feed_registry"`, which a caller cannot influence.

### 5.3. Component: verify

#### Description

The verify module is the security gate of the protocol. Its job is to decide whether an update was really signed by Stork's EVM key. It holds no state of its own, but every feed write depends on it.

The `verify_stork_evm_signature` function is the only public function. It builds the message that was signed, joins `r`, `s`, and `v` into a 65-byte signature, and hands both to the ECDSA check.

The `get_stork_message` function builds the exact bytes that Stork signs. The layout matches the EVM side: the 20-byte EVM public key, the 32-byte ID, and then the receive time and the value, each widened to a 32-byte big-endian word. The receive time is padded on the left with 24 zero bytes. The value is padded on the left with 16 sign bytes, which are `0xFF` when the value is negative and `0x00` when it is not:

```
// Left pad with 24 zero bytes
vector::append(&mut data, vector[0u8, 0, 0, /* ... 24 bytes total ... */ 0]);
// ... 8-byte big-endian recv_time ...
let sign_extension_byte = if (i128::is_negative(&quantized_value)) { 0xFF }
    else { 0x00 };
vector::append(&mut data, vector[sign_extension_byte, /* ... 16 bytes total ... */]);
let value_bytes = i128::to_bytes(quantized_value);
vector::append(&mut data, value_bytes);
vector::append(&mut data, publisher_merkle_root);
vector::append(&mut data, value_compute_alg_hash);
```

The result is hashed, then wrapped with the standard EIP-191 prefix \x19Ethereum Signed Message: \n32 before recovery.

The `verify_ecdsa_signature` function rejects anything that is not exactly 65 bytes, recovers the public key from the signature, turns it into an Ethereum address, and accepts only if that address exactly matches the configured key:

```
let evm_pubkey = get_eth_pubkey(&recovered_pubkey);
evm_pubkey == *pubkey
```

The `recover_secp256k1_pubkey` function accepts a `v` of 27 or 28, mapping them to recovery IDs 0 and 1, and rejects anything else:

```
let recovery_id = match (v) {
  27 => 0,
  28 => 1,
  _ => return option::none(),
};
```

The `get_eth_pubkey` function hashes the 64-byte public key with Keccak-256 and keeps the last 20 bytes, which is the Ethereum address.

## Invariants

- A signature is accepted only if it recovers to the configured Stork key. This holds. The recovered address is compared byte for byte against the stored key.
- Wrong-size signatures are rejected before any recovery happens. This holds. Anything that is not exactly 65 bytes returns false.
- Only the standard recovery values are accepted. This holds. Only a `v` of 27 or 28 is allowed; everything else fails.
- It should not be possible to turn one valid signature into a second valid one. This does not hold. There is no low-s check, so signatures are malleable. The impact is small; a reshaped signature still has to recover to the Stork key, and the freshness check would treat the resulting write as a duplicate.
- The metadata fields in the message should be the EVM contract's fixed 32-byte values. This does not hold. The Merkle root and alg hash are added at whatever length they arrive. See [Finding 3.2](#) for more detail.

## Test coverage

### Cases covered

- Full checks of a real positive-value signature and a real negative-value signature: `test_verify_stork_evm_signature` and

`test_verify_stork_evm_signature_negative_value`. The negative case exercises the 0xFF padding.

- The ECDSA check, the `r / s / v` packing, the message hash, the EIP-191 wrapping, and the key recovery each have their own tests against known values: `test_verify_ecdsa_signature`, `test_get_rsv_signature_from_parts`, `test_get_stork_message_hash`, `test_get_recoverable_message`, and `test_recover_secp256k1_pubkey`.
- The bad-signature rejection is also checked end to end by `test_feed_operations_invalid_signature`.

#### Cases not covered

- The wrong-length branch on its own.
- A `v` other than 27 or 28.
- A reshaped, high-`s` signature.
- A signature checked against a key that does not match, at the `verify` level.
- Metadata fields that are not 32 bytes.

#### Attack surface

- Every input comes from the caller, since they all come from the update payload. The protection is that each one is folded into the hashed message, so the recovered address only matches the Stork key if Stork actually signed that exact set of values. Changing any field breaks the recovery.
- The Merkle root and alg hash have no length limit, but they cannot be used to forge anything. They are folded into the signed message, so changing them changes the hash, and the signature still has to recover to the Stork key. They are also never stored or read after verification, so a wrong length has no on-chain effect. Enforcing a 32-byte length would be a defensive step that keeps the message in line with the EVM contract's `bytes32` fields and rejects malformed input early.
- The `v` handling only allows the Ethereum `personal_sign` values, so a bad `v` is rejected cleanly.
- The message does not include a chain ID or the `StorkState` object ID. This means a validly signed price can be submitted by anyone, on any chain. That is intended; the value is trustworthy no matter who relays it.

### 5.4. Component: Update inputs and encoded types

#### Description

This component is about how raw bytes from the caller turn into the typed values the update path uses, and the small value types that end up in the signed message. It covers

`update_temporal_numeric_value_evm_input`,  
`update_temporal_numeric_value_evm_input_vec`, `encoded_asset_id`, `evm_pubkey`, `i128`,

temporal\_numeric\_value, and temporal\_numeric\_value\_feed.

The `update_temporal_numeric_value_evm_input::new` function packs one update. It checks the ID and the value but stores the metadata and signature byte vectors exactly as they arrive:

```
UpdateTemporalNumericValueEvmInput {
    id: encoded_asset_id::from_bytes(id),
    temporal_numeric_value:
    temporal_numeric_value::new(temporal_numeric_value_timestamp_ns,
    i128::new(temporal_numeric_value_magnitude,
    temporal_numeric_value_negative)),
    publisher_merkle_root,
    value_compute_alg_hash,
    r,
    s,
    v,
}
```

The ID goes through `encoded_asset_id::from_bytes`, which requires 32 bytes, and the value goes through `i128::new`, which limits the magnitude. The `publisher_merkle_root`, `value_compute_alg_hash`, `r`, and `s` fields are stored with no length check at all.

The `update_temporal_numeric_value_evm_input_vec::new` function builds a batch. It rejects an empty input, and it requires all nine parallel vectors to be the same length before it builds the entries one index at a time:

```
assert!(ids.length() > 0, ENoUpdates);
assert!(ids.length() == temporal_numeric_value_timestamp_nss.length(),
    EInvalidLengths);
// ... one equal-length assertion per remaining field ...
assert!(ids.length() == vs.length(), EInvalidLengths);
```

The `encoded_asset_id::from_bytes` function requires a 32-byte ID, and `evm_pubkey::from_bytes` requires a 20-byte key.

The `i128::new` function limits the magnitude based on the sign and turns negative zero into positive zero:

```
if (!negative) {
    assert!(magnitude <= MAX_POSITIVE_MAGNITUDE, EMagnitudeTooLarge);
} else {
    assert!(magnitude <= MAX_NEGATIVE_MAGNITUDE, EMagnitudeTooLarge);
};
if (magnitude == 0) {
    negative = false;
};
```



Here, `MAX_POSITIVE_MAGNITUDE` is  $L!2^{127} - 1$  and `MAX_NEGATIVE_MAGNITUDE` is  $L!2^{127}$ . The `from_u128` function reads a two's-complement u128, and `to_bytes` writes the value back out as 16 big-endian two's-complement bytes for the signed message.

The `temporal_numeric_value` and `temporal_numeric_value_feed` modules are simple containers. The feed's `new` and `set_latest_value` are `public(package)`, so they can only be reached through the verified update path.

## Invariants

- Asset IDs are always exactly 32 bytes, which keeps the registry keys uniform. This holds, enforced by `encoded_asset_id::from_bytes`.
- EVM public keys are always exactly 20 bytes. This holds, enforced by `evm_pubkey::from_bytes`.
- A signed value never goes outside the `int128` range, and zero has a single form. This holds. The `i128::new` function limits the magnitude by sign and folds negative zero into positive zero.
- Batch building never reads past the end of any field vector. This holds. All nine vectors must be the same, nonzero length.
- The metadata fields should be the EVM contract's fixed 32-byte values. This does not hold. They are stored at any length. See Finding [3.2](#) for more detail.
- The `i128` comment should describe the sign field correctly. This does not hold. The comment says the opposite of what the code does. See Finding [3.5](#) for more detail.

## Test coverage

### Cases covered

- The `i128` module is well covered: magnitude limits for both signs, the sign accessors and their `EInvalidSign` aborts, the single form of zero, `from_u128` for both signs, and `to_bytes` for positive, negative, and zero.
- The `encoded_asset_id` and `evm_pubkey` modules each cover a valid build, the wrong-length abort, and reading the bytes back.
- The `update_temporal_numeric_value_evm_input_vec` module covers the empty-input abort, the length-mismatch abort, `length`, and a field-by-field `get_data` check.
- The `update_temporal_numeric_value_evm_input` module covers one happy-path build, and the container types cover their getters.

### Cases not covered

- Building an input with a non-32-byte Merkle root or alg hash or odd-length `r` and `s`. There is no check at this layer, so there is nothing to reject.
- A non-canonical signature split, where `r` and `s` are not 32 bytes each but `r || s || v` is still 65 bytes, so it reaches the recovery step.

- Round-trips of `from_u128` over `to_bytes` for a range of negative values.
- Large batches — the loop is only tested with up to four entries in the type tests and one entry end to end.

### Attack surface

- Every raw input comes from the caller. The ID and the EVM key are strictly length checked, so a malformed ID or key aborts right away instead of becoming a bad registry key. This is the main defense at the parsing layer.
- The metadata, `r`, and `s` fields accept any length, and they are checked differently downstream. The verifier does not check that `r` and `s` are 32 bytes each. It only checks that the joined `r || s || v` is exactly 65 bytes, so a noncanonical split, such as a 31-byte `r` and a 33-byte `s`, can still reach the recovery step as long as the total is 65 bytes. The recovered-address check then still has to match the Stork key, so this does not lead to a forgery. For the metadata fields, the only effect is on the bytes added to the message, so an odd length just leads to a failed signature check, not a forgery.
- The `i128` magnitude and sign are limited at build time, so an attacker cannot create an out-of-range value. The flipped comment is a documentation bug, not an exploitable one: the code and all the accessors consistently treat `negative == true` as negative.
- The batch-length fields must match and be nonzero, which removes any out-of-bounds or silent-truncation risk at the vector boundary. A large batch is still allowed, but it costs more, and the fee is charged per non-stale write.
- The container types can only be changed through `public(package)` setters reached from the verified update path, so they are not a separate way in.

## 6. Assessment Results

During our assessment on the scoped Stork Oracle modules, we discovered seven findings. No critical issues were found. Five findings were of low impact and the remaining findings were informational in nature.

---

### 6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.